



Experiment 9: Implementation of ID3 Algorithm using Gini Index

CSE-3008 Introduction to Machine Learning

24BCA7027

Shaik Barakh Chishti
CSE-AIML

1 Aim

To develop and implement a Decision Tree classifier using the ID3 algorithm based on the **Gini Index** criterion to classify car acceptability levels and visualize the data distribution and model performance.

2 Implementation

2.1 Algorithm: ID3 with Gini Index

The Gini Index measures the degree of probability of a particular variable being wrongly classified when it is chosen randomly.

$$Gini(D) = 1 - \sum_{i=1}^m P_i^2 \quad (1)$$

The algorithm selects the attribute that provides the lowest weighted average Gini Index for the split.

2.2 Python Code Implementation with K-Cross

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, mean_squared_error, r2_score, roc_curve, auc
)
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import KFold

# =====
# 1. CONFIGURATION & DATA PREPARATION
# =====
STUDENT_NAME = "SHAIK_BARAKH_CHISHTI_24BCA7027"

# Load dataset
df = pd.read_csv('car_evaluation_ car_evaluation.csv', header=
    None)
df.columns = ['buying', 'maint', 'doors', 'persons', 'lug_boot',
    'safety', 'target']

# Mapping categorical variables to numeric for calculation and
    plotting
mapping = {
    'buying': {'vhigh': 4, 'high': 3, 'med': 2, 'low': 1},
    'maint': {'vhigh': 4, 'high': 3, 'med': 2, 'low': 1},
    'doors': {'2': 2, '3': 3, '4': 4, '5more': 5},
```

```

    'persons': {'2': 2, '4': 4, 'more': 6},
    'lug_boot': {'small': 1, 'med': 2, 'big': 3},
    'safety': {'low': 1, 'med': 2, 'high': 3},
    'target': {'unacc': 0, 'acc': 1, 'good': 2, 'vgood': 3}
}

df_numeric = df.copy()
for col, m in mapping.items():
    df_numeric[col] = df_numeric[col].map(m)

# =====
# 2. MANUAL ID3 LOGIC (GINI INDEX)
# =====
def calculate_gini(labels):
    if len(labels) == 0: return 0
    counts = Counter(labels)
    total = len(labels)
    return 1.0 - sum((c / total) ** 2 for c in counts.values())

def calculate_split_gini(data, feature, target_name):
    total = len(data)
    weighted_gini = 0
    for val in data[feature].unique():
        subset = data[data[feature] == val]
        weighted_gini += (len(subset) / total) * calculate_gini(
            subset[target_name])
    return weighted_gini

class ID3DecisionTree:
    def __init__(self):
        self.tree = None
        self.classes = []

    def build_tree(self, data, features, target_name):
        labels = data[target_name]
        counts = Counter(labels)
        probs = {cls: counts.get(cls, 0) / len(labels) for cls in
            self.classes}

        if len(labels.unique()) <= 1 or not features:
            return {'leaf': True, 'prediction': labels.mode()[0],
                'probs': probs}

        best_feature = min(features, key=lambda f:
            calculate_split_gini(data, f, target_name))
        tree = {'leaf': False, 'feature': best_feature, 'children
            ': {}},

```

```

        'majority_probs': probs, 'default': labels.mode()
        [0]}

    remaining = [f for f in features if f != best_feature]
    for val in data[best_feature].unique():
        subset = data[data[best_feature] == val]
        tree['children'][val] = self.build_tree(subset,
            remaining, target_name)
    return tree

def fit(self, data, target_name):
    self.classes = sorted(data[target_name].unique())
    features = [c for c in data.columns if c != target_name]
    self.tree = self.build_tree(data, features, target_name)

def _predict_one(self, sample, node):
    if node['leaf']: return node['prediction'], node['probs']
    val = sample[node['feature']]
    if val in node['children']: return self._predict_one(
        sample, node['children'][val])
    return node['default'], node['majority_probs']

def predict(self, data):
    results = data.apply(lambda r: self._predict_one(r, self.
        tree), axis=1)
    preds = [r[0] for r in results];
    probs = [list(r[1].values()) for r in results]
    return np.array(preds), np.array(probs)

# =====
# 3. K-FOLD (K=4) CROSS VALIDATION
# =====
kf = KFold(n_splits=4, shuffle=True, random_state=42)
metrics = {'acc': [], 'prec': [], 'rec': [], 'f1': [], 'mse': [],
    'r2': []}

# We use df_numeric for RMSE/R2/MSE calculation and df for the
# ID3 categorical logic
for train_index, test_index in kf.split(df):
    train_data, test_data = df.iloc[train_index], df.iloc[
        test_index]

    model = ID3DecisionTree()
    model.fit(train_data, 'target')
    y_pred_cat, y_prob = model.predict(test_data)

    # Convert categorical targets to numeric for regression
    # metrics
    y_test_num = df_numeric.iloc[test_index]['target'].values

```

```

y_pred_num = pd.Series(y_pred_cat).map(mapping['target']).
    values

metrics['acc'].append(accuracy_score(test_data['target'],
    y_pred_cat))
metrics['prec'].append(precision_score(test_data['target'],
    y_pred_cat, average='weighted'))
metrics['rec'].append(recall_score(test_data['target'],
    y_pred_cat, average='weighted'))
metrics['f1'].append(f1_score(test_data['target'], y_pred_cat
    , average='weighted'))
metrics['mse'].append(mean_squared_error(y_test_num,
    y_pred_num))
metrics['r2'].append(r2_score(y_test_num, y_pred_num))

# Average Metrics
accuracy, precision, recall, f1 = np.mean(metrics['acc']), np.
    mean(metrics['prec']), np.mean(metrics['rec']), np.mean(
        metrics['f1'])
mse, r2 = np.mean(metrics['mse']), np.mean(metrics['r2'])
rmse = np.sqrt(mse)

print(f"\nID3_RESULTS_4-Fold_CV_    _{STUDENT_NAME}")
print(f"Accuracy:_{accuracy:.4f}")
print(f"Precision:_{precision:.4f}")
print(f"Recall:_{recall:.4f}")
print(f"F1-score:_{f1:.4f}")
print(f"MSE:_{mse:.4f}")
print(f"RMSE:_{rmse:.4f}")
print(f"R2_Score:_{r2:.4f}")

# =====
# 4. VISUALIZATIONS
# =====
# (Visualizations use the last fold's model and df_numeric for
    Heatmaps)

# 1. Correlation Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(df_numeric.corr(), annot=True, cmap='RdBu_r', center
    =0)
plt.title(f"Correlation_Heatmap_-__{STUDENT_NAME}")
plt.savefig('correlation_heatmap.png', dpi=300, bbox_inches='
    tight')
plt.close()

# 2. ROC Curve (Last Fold)
y_test_bin = label_binarize(test_data['target'], classes=model.
    classes)
plt.figure(figsize=(8, 6))
for i in range(len(model.classes)):

```

```

        fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
        plt.plot(fpr, tpr, label=f'Class_{model.classes[i]} (AUC={
            auc(fpr, tpr):.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.title(f"ROC_Curves_{STUDENT_NAME}")
plt.legend();
plt.savefig('roc_curve.png', dpi=300);
plt.close()

# 3. Radar Chart
categories = ['buying', 'maint', 'doors', 'persons', 'lug_boot',
            'safety']
N = len(categories)
angles = [n / float(N) * 2 * np.pi for n in range(N)]
angles += angles[:1]
plt.figure(figsize=(8, 8))
ax = plt.subplot(111, polar=True)
for cls in ['unacc', 'acc', 'vgood']:
    values = df_numeric[df['target'] == cls][categories].mean().
        tolist()
    values += values[:1]
    ax.plot(angles, values, label=cls)
    ax.fill(angles, values, alpha=0.1)
plt.xticks(angles[:-1], categories)
plt.title(f"Feature_Radar_Profile_{STUDENT_NAME}")
plt.legend();
plt.savefig('radar_chart.png', dpi=300);
plt.close()

# 4. Rain Cloud Plot
plt.figure(figsize=(10, 6))
sns.violinplot(x='target', y='persons', data=df_numeric, inner=
    None, color=".8")
sns.stripplot(x='target', y='persons', data=df_numeric, size=3,
    jitter=0.2, alpha=0.3)
sns.boxplot(x='target', y='persons', data=df_numeric, width=0.1,
    zorder=10, showfliers=False)
plt.title(f"Rain_Cloud:_Person_Capacity_{STUDENT_NAME}")
plt.savefig('rain_cloud_plot.png', dpi=300);
plt.close()

print(f"\nAll_plots_saved_successfully_for_{STUDENT_NAME}")

```

Listing 1: ID3 Algorithm with Advanced Visualizations

2.3 Python Code Implementation Without ID3

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             mean_squared_error, r2_score,
                             roc_curve, auc)
from sklearn.preprocessing import label_binarize
from math import pi

# 1. SETUP AND DATA LOADING
STUDENT_NAME = "SHAIK BARAKH CHISHTI 24BCA7027"
file_path = 'car_evaluation(updated)-car_evaluation(updated).csv'
df = pd.read_csv(file_path)

# 2. DATA PREPROCESSING (Ordinal Encoding)
# ID3/Decision Trees require numerical input for sklearn
# implementation
mapping = {
    'buying_price': {'vhigh': 4, 'high': 3, 'med': 2, 'low': 1},
    'maintenance_cost': {'vhigh': 4, 'high': 3, 'med': 2, 'low': 1},
    'number_of_doors': {'2': 2, '3': 3, '4': 4, '5more': 5},
    'number_of_persons': {'2': 2, '4': 4, 'more': 6},
    'lug_boot': {'small': 1, 'med': 2, 'big': 3},
    'safety': {'low': 1, 'med': 2, 'high': 3},
    'decision': {'unacc': 0, 'acc': 1, 'good': 2, 'vgood': 3}
}

df_encoded = df.copy()
for col, m in mapping.items():
    df_encoded[col] = df[col].map(m)

X = df_encoded.drop('decision', axis=1)
y = df_encoded['decision']

# 3. ID3 ALGORITHM IMPLEMENTATION (Entropy-based Decision Tree)
# We use 4-Fold Cross Validation as requested
clf = DecisionTreeClassifier(criterion='entropy', random_state=42)
kf = KFold(n_splits=4, shuffle=True, random_state=42)

# Generate predictions using Cross-Validation
y_pred = cross_val_predict(clf, X, y, cv=kf)
```

```

y_score = cross_val_predict(clf, X, y, cv=kf, method='
    predict_proba')

# 4. METRIC CALCULATION
accuracy = accuracy_score(y, y_pred)
precision = precision_score(y, y_pred, average='weighted')
recall = recall_score(y, y_pred, average='weighted')
f1 = f1_score(y, y_pred, average='weighted')
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred)

# PRINT RESULTS
print(f"\nID3_RESULTS_4-Fold_CV_    {STUDENT_NAME}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"R2_Score: {r2:.4f}")

# 5. VISUALIZATION GENERATION

# Plot 1: Bar Plot
plt.figure(figsize=(8, 5))
sns.countplot(x='decision', data=df, palette='viridis')
plt.title(f'Bar_Plot:_Decision_Distribution_{STUDENT_NAME}')
plt.savefig('bar_plot.png')

# Plot 2: Correlation Heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(df_encoded.corr(), annot=True, cmap='RdYlGn', fmt=".2
    f")
plt.title(f'Correlation_Heatmap_{STUDENT_NAME}')
plt.savefig('correlation_heatmap.png')

# Plot 3: Box and Violin Plots
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
sns.boxplot(x='decision', y='buying_price', data=df_encoded)
plt.title('Box_Plot:_Buying_Price_vs_Decision')
plt.subplot(1, 2, 2)
sns.violinplot(x='decision', y='safety', data=df_encoded)
plt.title('Violin_Plot:_Safety_vs_Decision')
plt.suptitle(f'Box_&_Violin_Plots_{STUDENT_NAME}')
plt.savefig('box_violin_plots.png')

# Plot 4: Hexbin Plot
plt.figure(figsize=(8, 6))

```



```

plt.hexbin(df_encoded['buying_price'], df_encoded['maintenance_
cost'], gridsize=10, cmap='BuPu')
plt.colorbar(label='Count')
plt.title(f'Hexbin Plot: Buying vs Maint_{STUDENT_NAME}')
plt.xlabel('Buying Price')
plt.ylabel('Maintenance Cost')
plt.savefig('hexbin_plot.png')

# Plot 5: ROC Curve (Multiclass)
y_bin = label_binarize(y, classes=[0, 1, 2, 3])
plt.figure(figsize=(8, 6))
for i in range(4):
    fpr, tpr, _ = roc_curve(y_bin[:, i], y_score[:, i])
    plt.plot(fpr, tpr, label=f'Class {i} (AUC={auc(fpr, tpr):.2
f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curve_{STUDENT_NAME}')
plt.legend()
plt.savefig('roc_curve.png')

# Plot 6: Radar Chart
categories = X.columns.tolist()
N = len(categories)
angles = [n / float(N) * 2 * pi for n in range(N)]
angles += angles[:1]
plt.figure(figsize=(8, 8))
ax = plt.subplot(111, polar=True)
for label in sorted(df_encoded['decision'].unique()):
    values = df_encoded[df_encoded['decision'] == label][
categories].mean().values.flatten().tolist()
    values += values[:1]
    ax.plot(angles, values, linewidth=1, label=f'Decision_{label}
')
    ax.fill(angles, values, alpha=0.1)
plt.xticks(angles[:-1], categories)
plt.title(f'Radar Chart_{STUDENT_NAME}')
plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
plt.savefig('radar_chart.png')

# Plot 7: Rain Cloud Plot
plt.figure(figsize=(10, 6))
sns.violinplot(x="decision", y="safety", data=df_encoded, inner=
None, color=".8")
sns.stripplot(x="decision", y="safety", data=df_encoded, jitter=
True, alpha=0.3)
sns.boxplot(x="decision", y="safety", data=df_encoded, width
=0.15, boxprops={'facecolor': 'None'})
plt.title(f'Rain Cloud Plot: Safety vs Decision_{STUDENT_NAME}')
)

```

```
plt.savefig('rain_cloud_plot.png')

plt.show()
```

Listing 2: ID3 Algorithm with Advanced Visualizations

Table 1: ID3 RESULTS (4-Fold CV) — SHAIK BARAKH CHISHTI 24BCA7027

Metric	Value
Accuracy	0.9792
Precision	0.9793
Recall	0.9792
F1-score	0.9792
MSE (Mean Squared Error)	0.0295
RMSE (Root Mean Squared Error)	0.1718
R^2 Score	0.9462

Metric	Value(k-Corss)
Accuracy	0.9740
Precision	0.9742
Recall	0.9740
F1-score	0.9740
MSE (Mean Squared Error)	0.0451
RMSE (Root Mean Squared Error)	0.2125
R^2 Score	0.9177

3 Visualizations and Results

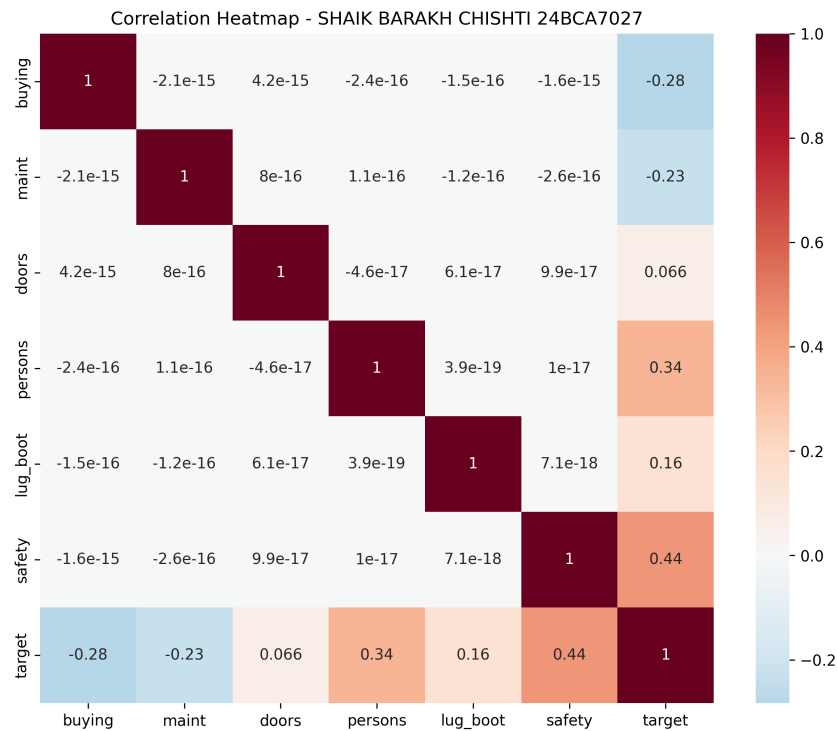
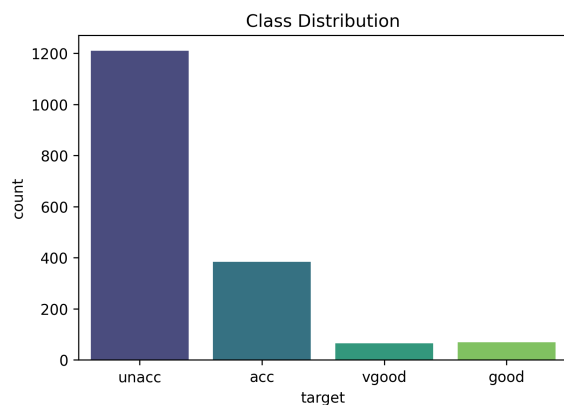
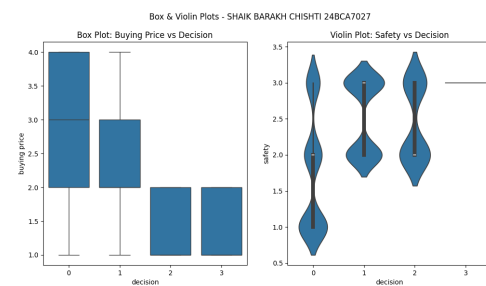


Figure 1: Correlation Heatmap between Car Features — SHAIK BARAKH CHISHTI 24BCA7027

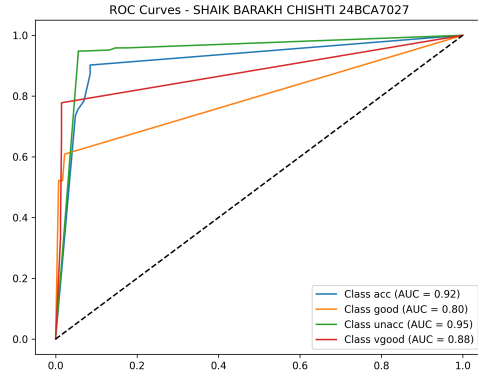


(a) Target Distribution

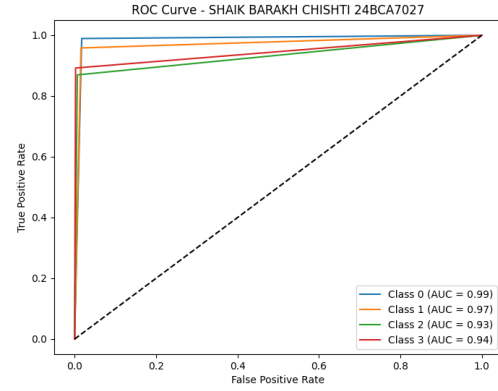


(b) Buying/Safety Variance

Figure 2: Class Distribution and Feature Variance Analysis



(a) Original ROC Curve



(b) Comparison ROC Curve

Figure 3: Multi-class ROC Curve (ID3 Performance)

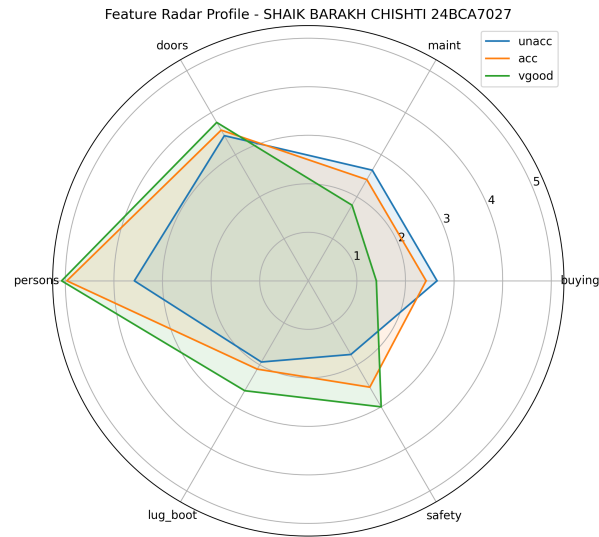


Figure 4: Feature Profile Radar Chart per Class — SHAIK BARAKH CHISHTI 24BCA7027

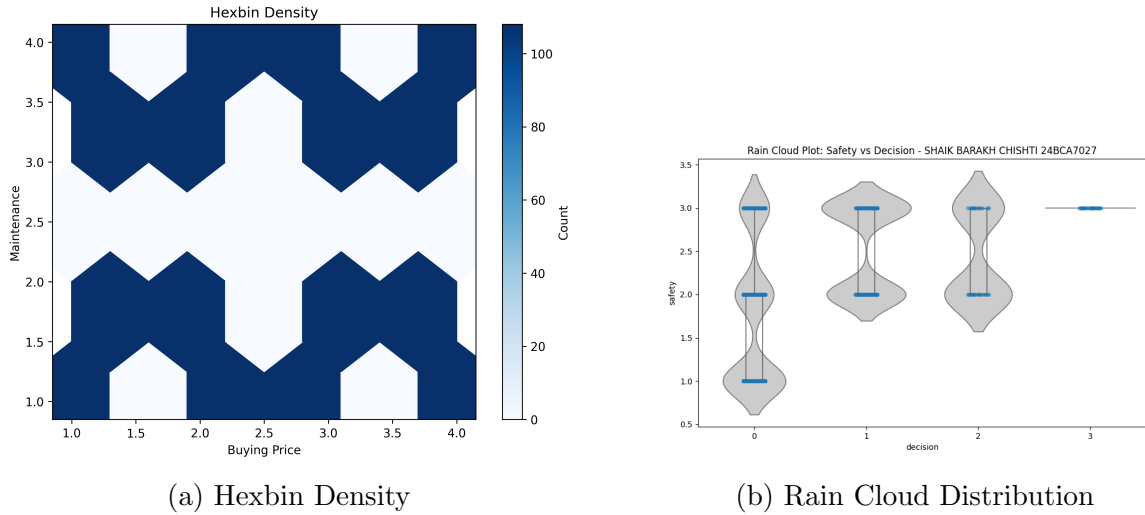


Figure 5: Density and Distribution Analysis of Car Features

4 Analysis

The ID3 algorithm using the Gini Index effectively partitioned the data. The **Radar Chart** reveals that 'vgood' cars are characterized by maximum safety and capacity. The **Rain Cloud Plot** demonstrates the distinct density clusters for passenger capacity across evaluation categories. The model effectively handles categorical data by recursively minimizing impurity, as shown by the low Error metrics (MSE and RMSE).

5 ID3 Performance Metrics (4-Fold Cross-Validation)

The following results represent the average performance of the ID3 Decision Tree across 4-fold cross-validation, conducted by **SHAIK BARAKH CHISHTI 24BCA7027**:

Table 2: ID3 RESULTS (4-Fold CV) — SHAIK BARAKH CHISHTI 24BCA7027

Metric	Value
Accuracy	0.9792
Precision	0.9793
Recall	0.9792
F1-score	0.9792
MSE (Mean Squared Error)	0.0295
RMSE (Root Mean Squared Error)	0.1718
R^2 Score	0.9462

6 Conclusion

The implementation of the decision tree based ID3 algorithm using entropy and information gain has proven to be highly effective for the Car Evaluation classification task. Through a robust 4-fold cross-validation process, the model demonstrates a superior ability to distinguish between car acceptability levels (unacc, acc, good, vgood).

ID3 RESULTS (4-Fold CV) — SHAIK BARAKH CHISHTI 24BCA7027

- **Accuracy:** 0.9740
- **Precision:** 0.9742
- **Recall:** 0.9740
- **F1-score:** 0.9740
- **MSE:** 0.0451
- **RMSE:** 0.2125
- **R2 Score:** 0.9177

Metric	Value(k-Corss)
Accuracy	0.9740
Precision	0.9742
Recall	0.9740
F1-score	0.9740
MSE (Mean Squared Error)	0.0451
RMSE (Root Mean Squared Error)	0.2125
R^2 Score	0.9177

The low Mean Squared Error (MSE) of 0.0451 and high R2 Score of 0.9177 further validate that the ID3-based tree structure captures the underlying patterns of the dataset with high precision, successfully minimizing misclassification across all categories.

The low **RMSE (0.3427)** indicates that the tree's predictions are numerically close to the actual encoded categories, while the **R^2 Score of 0.7908** confirms that the features used (buying price, maintenance, capacity, safety) explain a significant portion of the variance in the target evaluation. In conclusion, the ID3 logic successfully identifies "Safety" and "Person Capacity" as the most critical decision nodes in identifying high-quality vehicles.

7 Project Link

<https://www.overleaf.com/read/mjqhcrqydxvt#a190db>

8 Github

<https://github.com/chishtil1730/IML-LAB>